

UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnicatura Universitaria en Programación a Distancia

BASE DE DATOS II

Unidad 1 · Integridad, Transacciones y Concurrency — Semana 2

Trabajo Práctico de Laboratorio: Concurrency e IA como motor primario

Consigna para el alumno · trabajar sobre el esquema del propio proyecto integrador

Cómo usar esta guía

Este TP pone en práctica la teoría de la Semana 2 (bloqueos, MVCC, interbloqueos y diagnóstico) y, al mismo tiempo, opera el eje transversal de la Unidad 1: la IA como motor primario del trabajo con datos. A diferencia de los documentos teóricos —que usan un esquema genérico de ilustración—, acá se trabaja directamente sobre el esquema del propio proyecto integrador, con OpenCode y Kiro como herramientas y con el protocolo de seguridad de la cátedra como condición no negociable en cada paso.

Regla de fondo: se delega la escritura, nunca la decisión. Todo lo que la IA proponga en este TP se lee, se prueba sobre una copia y dentro de una transacción, y se defiende oralmente antes de darlo por bueno.

1. Objetivos de la práctica

- Reproducir, con dos sesiones concurrentes sobre la base del proyecto, al menos tres de las anomalías y escenarios de concurrency vistos en la Semana 2, y verificar en el motor real qué nivel de aislamiento o qué mecanismo de bloqueo los evita.
- Generar, con OpenCode, restricciones de integridad para reglas de negocio propias del proyecto, versionadas en Git, con revisión de diff línea por línea antes de aplicarlas.
- Aplicar el protocolo de seguridad de la cátedra (copia, transacción, respaldo) como procedimiento estándar, no como excepción, en cada operación que un agente de IA proponga sobre la base.
- Reconocer, a partir de casos reales, por qué ningún script generado por IA se ejecuta sin leerse antes — y practicar esa lectura crítica sobre un script deliberadamente peligroso.
- Producir una Declaración de Uso de IA (DUIA) completa para cada ejercicio, como parte integral de la entrega, no como trámite posterior.

2. Antes de empezar — requisitos previos

Requisito	Detalle
Esquema propio aplicado	El schema.sql de tu proyecto (Semana 1) corriendo sobre una base de trabajo local — nunca sobre datos reales de terceros.
Repositorio Git inicializado	Al menos un commit con el esquema y los datos de carga inicial ya versionados.
OpenCode instalado y autenticado	Ver la fuente formativa recomendada en la sección 8 si todavía no lo configuraste.
Kiro instalado	Con el proyecto abierto al menos una vez, para que exista la carpeta .kiro/ en el repo.

Requisito	Detalle
Dos conexiones a la base	Dos pestañas de psql, o dos conexiones abiertas en tu cliente SQL (DBeaver, etc.), listas para simular dos sesiones concurrentes.

3. Parte 0 — Puesta a punto: repositorio, IA y protocolo de seguridad

Antes de tocar la base con un script generado, hay que dejar dos cosas en su lugar: el flujo de trabajo con las herramientas de IA de la cátedra, y el protocolo que va a proteger los datos de cualquier error — propio o del agente.

3.1. OpenCode y Kiro sobre el repositorio de scripts

OpenCode es un agente de codificación de terminal: se le pide algo en lenguaje natural y trabaja directamente sobre los archivos del repositorio. Para este TP conviene usarlo en su modo Plan antes que en modo directo: Plan describe qué va a hacer sin tocar ningún archivo todavía, y recién después de revisar el plan se le pide que lo ejecute.

```
cd tu-proyecto/
opencode
/init          # genera AGENTS.md describiendo el repo – comitealo

<TAB>         # cambia a modo Plan: describe, no modifica archivos
Necesito una restricción que impida ...

<TAB>         # de vuelta a modo Build, recién ahí aplica los cambios
```

Kiro conviene para cambios que ameritan una spec propia: requisitos, diseño y tareas antes de escribir código. Para este TP alcanza con generar los steering docs una vez, al principio, para que Kiro entienda las convenciones del esquema (nombres de tablas, patrón de borrado lógico, tipos ENUM del proyecto).

3.2. El protocolo de tres pasos: copia, transacción, respaldo

Es la condición de seguridad de la cátedra para cualquier script —propio o generado— que toque la base. Se aplica siempre, sin excepción, incluso cuando el cambio parece trivial.

Paso	Qué significa en la práctica	Cuándo se salta (nunca)
Copia	Se trabaja sobre una base de desarrollo, nunca sobre la que contiene datos que importan. <code>createdb -T plantilla_base copia_trabajo</code> .	No aplica: siempre hay copia.
Transacción	Todo script que escribe corre primero dentro de <code>BEGIN; ...; ROLLBACK;</code> para inspeccionar el efecto (filas afectadas, mensajes) antes de confirmar nada.	No aplica: siempre <code>BEGIN</code> antes de escribir.
Respaldo	<code>pg_dump</code> de la copia de trabajo antes de aplicar un cambio estructural (<code>ALTER</code> , <code>DROP</code> , migración), para poder volver atrás sin depender del <code>ROLLBACK</code> .	No aplica: siempre respaldo antes de DDL.

Entregable de la Parte 0

Un archivo protocolo_seguridad.md en la raíz del repo, con los tres pasos anteriores adaptados a los comandos concretos de tu entorno (motor, forma de crear la copia, dónde vive el respaldo). Se commitea antes de avanzar a la Parte 1 — sin este archivo no se puede continuar.

4. Parte 1 — Primer ejercicio con OpenCode: integridad versionada

El objetivo es generar, con OpenCode, restricciones declarativas o triggers para reglas de negocio de tu propio proyecto que hoy no están garantizadas por el motor — y hacerlo con un flujo que deje rastro y se pueda defender.

4.1. Consigna

Elegí entre dos y tres reglas de negocio de tu proyecto que hoy dependen de que «alguien se acuerde» de validarlas en la aplicación, y que podrían garantizarse en el motor. Ejemplos de reglas de este tipo (adaptalas a tu dominio): un rango de fechas coherente, una transición de estado válida (no volver de CONFIRMADO a PENDIENTE), un descuento que nunca supere el precio.

4.2. Flujo obligatorio

1. **Escribí la spec antes de pedir nada.** Una o dos frases por regla, con el nombre exacto de tabla y columna. Una spec ambigua produce un script ambiguo.
2. **Generá con OpenCode en modo Plan primero.** Revisá el plan que propone antes de dejarlo escribir un solo archivo.
3. **Leé el diff completo, línea por línea.** git diff antes de aplicar nada sobre la copia de trabajo. Si una línea no se entiende, no se aplica hasta entenderla.
4. **Aplicá dentro de una transacción sobre la copia.** BEGIN, aplicar, probar con INSERT válidos e inválidos, y recién ahí COMMIT.
5. **Commiteá con mensaje descriptivo.** El mensaje de commit debe decir qué regla de negocio se garantiza, no solo «agrego constraint».
6. **Completá la DUIA.** Con la tabla del punto siguiente, en el mismo commit o en un archivo aparte del repo.

Plantilla de Declaración de Uso de IA (DUIA)

Campo	Completar
Herramienta	OpenCode (modelo/proveedor configurado)
Spec o prompt utilizado	Texto exacto de la consigna dada a la IA
Qué generó	Resumen de los archivos y líneas que propuso
Qué se aceptó	Qué quedó tal cual lo generó la IA
Qué se modificó o descartó, y por qué	Cualquier corrección hecha a mano, con la razón
Verificación realizada	Los INSERT de prueba (válidos e inválidos) y su resultado

4.3. Defensa oral

En la clase siguiente, el docente elige al azar una línea del diff commiteado y pregunta qué hace esa línea puntual y qué pasaría si se sacara. No poder responderlo equivale a no haber hecho el trabajo, según la política de uso responsable de IA de la cátedra.

Entregable de la Parte 1

El script de restricciones commiteado en el repo, el historial de commits correspondiente (git log), y la DUIA completa. Se defiende oralmente en la clase siguiente.

5. Parte 2 — Laboratorio: anomalías con dos sesiones concurrentes

Acá se reproduce, sobre la base del propio proyecto y con dos sesiones abiertas al mismo tiempo, lo que en la teoría de la Semana 1 y 2 quedó descripto con ejemplos genéricos. La consigna pide, además, algo más exigente que reproducir el fenómeno: reconstruirlo con ayuda de IA y verificar esa reconstrucción contra el motor real.

5.1. Consigna general

Elegí y reproducí al menos tres de los siguientes cuatro escenarios sobre tablas de tu propio esquema:

- **Lectura no repetible.** Con Read Committed, mostrar que una misma consulta repetida dentro de la transacción cambia de resultado; repetir en Repeatable Read y mostrar que no cambia.
- **Lectura fantasma.** Un COUNT o SUM repetido dentro de la misma transacción, mientras otra sesión inserta una fila nueva que cumple la condición del WHERE.
- **Espera por bloqueo.** Dos sesiones pidiendo FOR UPDATE sobre la misma fila; la segunda debe quedar esperando hasta que la primera haga COMMIT o ROLLBACK.
- **Interbloqueo real (opcional, nota adicional).** Dos sesiones tomando dos filas en orden cruzado, hasta que el motor aborta a una con el error 40P01.

5.2. Reconstrucción con IA, verificada en el motor

Para cada escenario reproducido, el flujo es siempre el mismo:

1. **Reproducir el escenario** con las dos sesiones, capturando los comandos exactos y lo que devolvió cada uno.
2. **Pedirle a la IA una explicación** de qué pasó y qué nivel de aislamiento (o qué mecanismo de bloqueo) lo evitaría. Guardar esa explicación tal como la dio.
3. **Verificar esa explicación en el motor real** repitiendo el experimento con SET TRANSACTION ISOLATION LEVEL en el nivel que la IA propuso, y confirmando que el resultado efectivamente cambia como se esperaba.
4. **Registrar si la IA acertó.** Si la explicación no se confirma en el motor, esa discrepancia se documenta — no se descarta ni se oculta.

Este paso es la aplicación directa del criterio de la cátedra frente a la IA: se le pide una explicación, pero la que vale es la que confirma el motor, no la que da el modelo.

5.3. Informe

El informe se arma como un archivo informe_concurrencia.md con una sección por escenario, siguiendo esta estructura:

Campo	Contenido
Escenario	Cuál de los cuatro se reprodujo
Cómo se reprodujo	Comandos exactos de Sesión A y Sesión B, en orden
Qué se observó	Salida real de cada comando
Explicación de la IA	Copiada tal cual, con la herramienta usada
Verificación en el motor	Qué se repitió y qué resultado dio
Conclusión	Si la explicación de la IA se confirmó, y qué nivel/mecanismo resuelve el problema

Entregable de la Parte 2

El archivo informe_concurrencia.md con al menos tres escenarios completos, commitado en el repo junto con su DUIA correspondiente.

6. Parte 3 — El riesgo fundacional: por qué se lee antes de ejecutar

El protocolo de copia, transacción y respaldo de la Parte 0 no es burocracia: existe porque, cuando falla, el costo se mide en datos reales que no vuelven. Esta parte trabaja con casos documentados y con un ejercicio propio de lectura crítica.

6.1. Casos reales

Caso	Qué pasó
Replit — julio 2025	Un agente de codificación tenía instrucciones explícitas de no tocar la base de producción durante un «congelamiento» de código. Igual ejecutó comandos destructivos y borró registros de más de 1.200 ejecutivos y 1.100 empresas. Consultado por la recuperación, el agente afirmó que era imposible revertir el cambio; el usuario lo revirtió igual, a mano.
Google Gemini CLI — julio 2025	Al reorganizar archivos, el agente asumió que una operación había funcionado sin confirmarlo, y encadenó los pasos siguientes sobre una carpeta que en los hechos no existía, destruyendo archivos reales del usuario en el proceso.
Incidente «PocketOS» — agente con credenciales heredadas	Un agente que había heredado credenciales elevadas de un ingeniero borró una base de producción y sus copias de respaldo en cuestión de segundos, pese a una instrucción explícita de no ejecutar nada.
Confusión de entorno	Un desarrollador le pidió a un agente limpiar datos de un entorno de prueba; el agente se conectó, sin ningún error técnico, a la base de producción real y borró millones de filas de datos de clientes.

6.2. El patrón común

En ninguno de los cuatro casos el modelo «alucinó» código inválido ni fue víctima de un ataque externo: la sintaxis fue correcta y la intención, razonable. Lo que falló fue el paso que un humano atento habría hecho antes de ejecutar: confirmar contra qué base se estaba corriendo, leer el efecto real del comando, y no confiar en el reporte del propio agente sobre lo que había hecho. Es exactamente lo que el protocolo de la Parte 0 obliga a

interponer: una copia para que el error no toque nada real, una transacción para poder inspeccionar antes de confirmar, y un respaldo independiente para cuando ni siquiera eso alcanza.

6.3. Ejercicio de lectura crítica

Antes de ejecutar cualquiera de los siguientes dos scripts (ambos, supuestamente, generados para «dar de baja registros vencidos» sobre el esquema genérico de cátedra), identifiqué qué haría cada uno realmente y reescribílo corregido.

Script 1

```
-- Generado para: dar de baja las funciones de películas retiradas de cartel
UPDATE funcion
SET activa = FALSE;
```

Script 2

```
-- Generado para: limpiar las categorías sin productos asociados
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);
```

Para cada script, entregá: qué filas afectaría realmente tal como está escrito, por qué eso no coincide con la consigna que dice cumplir, y la versión corregida (con su WHERE, o con el manejo correcto de NULL en la subconsulta, según corresponda).

Entregable de la Parte 3

Un archivo ejercicio_lectura_critica.md con el análisis y la corrección de ambos scripts, commitado en el repo.

7. Entregables y rúbrica

Entregable	Dónde vive	Qué se evalúa
protocolo_seguridad.md	Repositorio Git	Que los tres pasos estén adaptados al entorno real del alumno, no copiados genéricamente.
Script de restricciones + DUJA	Repositorio Git (commit dedicado)	Calidad de la spec, prueba con casos válidos e inválidos, y la defensa oral del diff.
informe_concurrencia.md + DUJA	Repositorio Git	Que la explicación de la IA haya sido efectivamente verificada en el motor, no solo transcripta.
ejercicio_lectura_critica.md	Repositorio Git	Que identifique el efecto real de cada script antes de corregirlo, no solo que entregue la versión corregida.
Defensa oral en clase	Presencial / videollamada	Proceso tanto como producto: se puede explicar cada decisión tomada, propia y de la IA.

8. Fuente formativa recomendada

Para seguir un ejemplo guiado, paso a paso, de cada herramienta antes de aplicarla sobre tu propio proyecto:

- **OpenCode** — **Guía de introducción oficial:** opencode.ai/docs. Cubre instalación, configuración del proveedor, el comando /init que genera AGENTS.md, y el modo Plan que se usa en la Parte 1 de este TP.
- **Kiro** — **Documentación oficial:** kiro.dev/docs, y en particular la guía «Your first project» en kiro.dev/docs/getting-started/first-project, que recorre steering, specs y hooks sobre un proyecto real.

Ambas herramientas ya se instalaron y configuraron en la Semana 1 (puesta a punto de OpenCode, Kiro y Git); esta sección es para repasar el flujo antes de aplicarlo a un caso propio, no para instalar desde cero.

Checklist final antes de entregar

- ☐ protocolo_seguridad.md commiteado y aplicado en cada paso posterior.
- ☐ Cada script generado por IA fue leído línea por línea antes de aplicarse.
- ☐ Cada aplicación sobre la copia de trabajo pasó primero por BEGIN ... ROLLBACK.
- ☐ Las tres DUIA (Parte 1, Parte 2, Parte 3) están completas, no en blanco.
- ☐ Podés explicar, sin mirar el archivo, qué hace cada línea que commiteaste.